LeetCode Practice

6 Hard: Backtracking

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

6
HARD

Patterns (in order)

1. Backtracking (6)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

6 questions | tap a title to jump to it

Hard questions (6) Go >

Backtracking (6) Go >

H 1. Sudoku Solver LC 37

H 2. N-Queens LC 51

H 3. N-Queens II LC 52

H 4. Word Ladder II LC 126

H 5. Remove Invalid Parentheses LC 301

H 6. Number of Squareful Arrays LC 996

Hard questions (6)

HARD | TOPIC 1 OF 1

Backtracking

6 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Sudoku Solver LC 37
2. N-Queens LC 51
3. N-Queens II LC 52
4. Word Ladder II LC 126
5. Remove Invalid Parentheses LC 301
6. Number of Squareful Arrays LC 996

HARD | Backtracking | LeetCode #37 | 1 of 6 | 1/6

Sudoku Solver

QUESTION

Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy all of the following rules:

CONSTRAINTS

- 1. Each of the digits 1-9 must occur exactly once in each row.
- 2. Each of the digits 1-9 must occur exactly once in each column.
- 3. Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.

The '.' character indicates empty cells.

Example 1

Input

```
board = [["5","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".","."],[".","9","8",".",".",".",".","6","."],["8",".",".",".","6",".",".",".","3"],["4",".",".","8",".","3",".",".","1"],["7",".",".",".","2",".",".",".","6"],[".","6",".",".",".",".","2","8","."],[".",".",".","4","1","9",".",".","5"],[".",".",".",".","8",".",".","7","9"]]
```

Output

```
[["5","3","4","6","7","8","9","1","2"],["6","7","2","1","9","5","3","4","8"],["1","9","8","3","4","2","5","6","7"],["8","5","9","7","6","1","4","2","3"],["4","2","6","8","5","3","7","9","1"],["7","1","3","9","2","4","8","5","6"],["9","6","1","5","3","7","2","8","4"],["2","8","7","4","1","9","6","3","5"],["3","4","5","2","8","6","1","7","9"]]
```

Explanation

The input board is shown above and the only valid solution is shown below:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

QUESTION

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

- board.length == 9
- board[i].length == 9
- board[i][j] is a digit or '.'.
- It is guaranteed that the input board has only one solution.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Fill empty cells ('.' characters) in a 9×9 Sudoku so each row, column,

and 3×3 box contains 1-9 exactly once. Exactly one valid solution exists. Right?"

"A few quick questions:

Modify in-place? Yes — no return value needed. Guaranteed unique solution? Yes."

"Let me walk: the standard 9×9 puzzle -> unique completed board ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def solve():
    pick the next empty cell
    for digit 1 to 9:
        if placing it keeps the board valid
            (scan its row, column and box):
            place it, recurse, undo if it fails
```

TIME	SPACE
O(9^m)	O(m)

WHY IT IS SLOW

m empty cells, up to 9 choices each, and each placement re-validates by scanning. Keeping row, column and box bitmasks and choosing the most constrained cell first prunes almost everything.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is re-scanning rows/cols/boxes for each validity check.

Pre-build three 9x9 boolean arrays: rows[r][d], cols[c][d], boxes[box_id][d].

Each check/update is O(1) instead of O(9).

`box_id = (r//3)*3 + c//3.`

Time: still O(9^m) worst case but dramatically faster in practice.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _37_SudokuSolver:
    def solveSudoku(self, board):
        rows = [[False]*10 for _ in range(9)]
        cols = [[False]*10 for _ in range(9)]
        boxes = [[False]*10 for _ in range(9)]
        empty = []
        for r in range(9):
            for c in range(9):
                if board[r][c] != '.':
                    d = int(board[r][c])
                    bid = (r//3)*3 + c//3
                    rows[r][d] = cols[c][d] = boxes[bid][d
] = True
                else:
                    empty.append((r, c))

        def backtrack(idx):
            if idx == len(empty): return True
            r, c = empty[idx]; bid = (r//3)*3 + c//3
            for d in range(1, 10):
                if not rows[r][d] and not cols[c][d] and n
ot boxes[bid][d]:
                    rows[r][d] = cols[c][d] = boxes[bid][d
] = True
                    board[r][c] = str(d)
                    if backtrack(idx + 1): return True
                    rows[r][d] = cols[c][d] = boxes[bid][d
] = False
                    board[r][c] = '.'
            return False
        backtrack(0)
```

PHASE 5 TEST & DEBUG

"Let me trace solveSudoku on a partially filled 9×9 board."

Find first empty cell (r,c). Try digits 1-9: check isValid(board,r,c,d).

isValid: no d in row r, no d in col c, no d in 3×3 box (r//3*3, c//3*3).

Place d -> recurse -> if returns True: done. Else undo (backtrack).

Example: empty at (0,0). Try '1': conflict in row? No. Col? No. Box? Check (0,0)-(2,2). If no conflict: place '1'. Next empty at (0,1). Try '1': same row -> conflict. Try '2'... etc.

When all 81 cells filled -> return True. Backtrack propagates True up call stack.

Key insight: constraint satisfaction — each placed digit constrains later choices; no valid digit -> backtrack.

O(9^81) worst case but pruning via constraint checks makes it fast in practice.

PHASE 5 TEST & DEBUG

Final answer confirmed ok

Trace: board[0][0]="5",r=0,c=0 -> seen_rows[0]={5}, seen_cols[0]={5}, seen_boxes[0]={5} — first cell passes; continue to (0,1).

PHASE 6 COMPLEXITY & FOLLOW-UP

O(9^m) worst case where m = empty cells. In practice, constraint propagation limits branching.

Space O(1): 27 boolean arrays + recursion O(m) depth.

Follow-up: Valid Sudoku (#36) — just check validity, no solving needed.

Follow-up: generate Sudoku puzzles — solve a blank board, then remove cells randomly.

IN YOUR APP | Expose | Editorial | Community

HARD | Backtracking | LeetCode #51 | 2 of 6 | 2/6

N-Queens

QUESTION

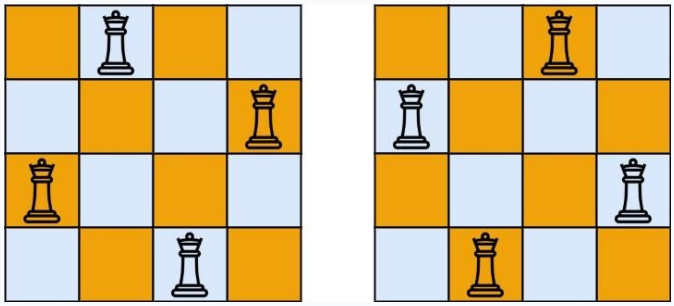
The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other.

Given an integer n, return all distinct solutions to the n-queens puzzle. You may return the answer in any order.

Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Example 1

Input	`n = 4`
Output	`[[".Q..","...Q","Q...","..Q."],["..Q.","Q...","...Q",".Q.."]]`
Explanation	*There exist two distinct solutions to the 4-queens puzzle as shown above*



Example 2

Input	`n = 1`
Output	`[["Q"]]`

CONSTRAINTS

- 1 <= n <= 9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Place n queens on an n×n chessboard so no two queens share a row, column, or diagonal.

Return all distinct solutions. Each solution is a list of board rows as strings. Right?"

"A few quick questions:

Return in any order? Yes. n <= 9 per constraints."

"Let me walk: n=4 -> 2 solutions: ['.Q..','...Q','Q...','..Q.'] and ['..Q.','Q...','...Q','.Q..'] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to put one queen in each row (n^n placements)
    keep it if no two queens share a column or
        diagonal
```

TIME	SPACE
O(n^n * n^2)	O(n)

WHY IT IS SLOW

Checks conflicts only after a full placement. Backtracking row by row with column and diagonal sets abandons a branch the moment a queen is attacked: about O(n!).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is generating all permutations before filtering.

Backtracking prunes invalid branches immediately:

Place one queen per row. Track occupied columns and both diagonals using sets.

When we find a safe column, recurse to the next row; backtrack by removing the queen.

Time: O(n!) — same worst case but with heavy early pruning. Space: O(n) sets + call depth."

Key insight: tracking occupied columns and both diagonal families as sets makes each validity check O(1) — backtracking on first conflict is faster than checking all n² positions.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _51_NQueens:
    def solveNQueens(self, n):
        result = []
        cols = set(); pos_diag = set(); neg_diag = set()
        board = [['.' for _ in range(n)] for _ in range(n)
]

        def backtrack(row):
            if row == n:
                result.append([''.join(r) for r in board])
; return
            for col in range(n):
                if col in cols or (row-col) in pos_diag or
 (row+col) in neg_diag:
                    continue
                cols.add(col); pos_diag.add(row-col); neg_
diag.add(row+col)
                board[row][col] = 'Q'
                backtrack(row + 1)
                cols.discard(col); pos_diag.discard(row-co
l); neg_diag.discard(row+col)
                board[row][col] = '.'
        backtrack(0)
```

```
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace N-Queens backtracking decision-by-decision for n=4."

We place exactly one queen per row. A position (r,c) is valid iff column c and both diagonals are free.

Trace:

Row 0:

`try col 0: SAFE -> place Q(0,0)`

Row 1:

`try col 0: NOT safe (same column) -> reject`

`try col 1: NOT safe (main diagonal) -> reject`

`try col 2: SAFE -> place Q(1,2)`

Row 2:

`try col 0: NOT safe (same column) -> reject`

`try col 1: NOT safe (anti diagonal) -> reject`

`try col 2: NOT safe (same column, main diagonal) -> reject`

`try col 3: NOT safe (main diagonal) -> reject`

backtrack from Q(1,2)

`try col 3: SAFE -> place Q(1,3)`

Row 2:

`try col 0: NOT safe (same column) -> reject`

`try col 1: SAFE -> place Q(2,1)`

Row 3:

`try col 0: NOT safe (same column, anti diagonal) -> reject`

`try col 1: NOT safe (same column, anti diagonal) -> reject`

`try col 2: NOT safe (main diagonal) -> reject`

`try col 3: NOT safe (same column, main diagonal) -> reject`

backtrack from Q(2,1)

`try col 2: NOT safe (main diagonal, anti diagonal) -> reject`

`try col 3: NOT safe (same column) -> reject`

backtrack from Q(1,3)

backtrack from Q(0,0)

`try col 1: SAFE -> place Q(0,1)`

Row 1:

`try col 0: NOT safe (anti diagonal) -> reject`

`try col 1: NOT safe (same column) -> reject`

`try col 2: NOT safe (main diagonal) -> reject`

`try col 3: SAFE -> place Q(1,3)`

Row 2:

`try col 0: SAFE -> place Q(2,0)`

Row 3:

`try col 0: NOT safe (same column) -> reject`

`try col 1: NOT safe (same column, main diagonal, anti diagonal) -> reject`

`try col 2: SAFE -> place Q(3,2)`

PHASE 5 TEST & DEBUG

FOUND solution at row 4 ok

Output (first solution found):

.Q..

...Q

Q...

..Q.

ok

path=[Q(0,1),Q(1,3),Q(2,0),Q(3,2)] -> first valid 4-queen solution. backtrack(row=1,path=[Q(0,1)]): try col=0: diagonal |0-1|=|1-0|=1 -> conflict -> skip (prune). try col=2: safe->path=[Q(0,1),Q(1,2)],call(row=2).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n!) heavy pruning reduces branching at each row	**O(n)** sets + O(n) board

Follow-up: N-Queens II (#52) — count solutions only; skip board construction.

Follow-up: for large n, use bitmask to represent cols/diagonals for ultra-fast pruning.

HARD | Backtracking | LeetCode #52 | 3 of 6 | 3/6

N-Queens II

QUESTION

The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other.

Given an integer n, return the number of distinct solutions to the n-queens puzzle.

Example 1	
Input	n = 4
Output	2
Explanation	*There are two distinct solutions to the 4-queens puzzle as shown.*

Example 2	
Input	n = 1
Output	1

CONSTRAINTS

- 1 <= n <= 9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count the number of distinct solutions to the n-queens puzzle (just the count, not the boards). Right?"

"A few quick questions:

Same constraint as N-Queens I: no two queens share row, column, or diagonal."

"Let me walk: n=4 -> 2 solutions ok"

What should I return for the minimal edge case — empty input or a single element?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to put one queen in each row (n^n placements)
    count it if no two queens share a column or
        diagonal
```

TIME	SPACE
O(n^n * n^2)	O(n)

WHY IT IS SLOW

Checks conflicts only after a full placement. Backtracking with column and diagonal sets prunes early: about O(n!).

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Backtracking counting solutions — O(n!). Walk-through:"

Track cols, diag1, diag2 as sets (same as N-Queens I).

Place queens row by row:

For each col in 0..n-1:

If col not in cols AND (row-col) not in diag1 AND (row+col) not in diag2:

Mark all three. Recurse to next row.

If reached row n: count++.

Unmark all three.

Return count.

WHY same structure as N-Queens: identical constraint checks — only change is

incrementing a counter instead of recording board configurations.

Diagonal key: row-col is unique for each '/' diagonal; row+col for each '\' diagonal.

Time: O(n!). Space: O(n) recursion + O(n) sets.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _52_NQueensII:
    def totalNQueens(self, n):
        count = [0]
        full_mask = (1 << n) - 1
        def backtrack(row, cols, diag1, diag2):
            if row == n:
                count[0] += 1
                return
            available = full_mask & ~(cols | diag1 | diag2
)
            while available:
                pos = available & (-available)
                available -= pos
                backtrack(row + 1, cols | pos, (diag1 | po
s) << 1, (diag2 | pos) >> 1)
        backtrack(0, 0, 0, 0)
        return count[0]
```

PHASE 5 TEST & DEBUG

"Let me trace totalNQueens n=4."

Same logic as Q51 but count solutions instead of storing boards.

Row 0, col 0: -> Row 1, col 2 -> Row 2: no valid -> backtrack.

Row 0, col 0 -> Row 1, col 3 -> Row 2, col 1 -> Row 3: col 3(clash), col 0(diag with (2,1)->|2-3|=1,|1-0|=1 yes diag), so no valid. Backtrack.

Actually for n=4 there are exactly 2 solutions:

Solution 1: queens at (0,1),(1,3),(2,0),(3,2).

Solution 2: queens at (0,2),(1,0),(2,3),(3,1).

Return 2 ok

PHASE 5 TEST & DEBUG

WHY count not store: saves memory since only the count matters.

path=[Q(r,c)...] builds queen positions. backtrack(row,path,cols,diag1,diag2): try each col -> if safe: add to path, recurse. prune: skip if col/diagonal conflict. count valid paths.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n!) with small constant via bitmasks	**O(n)** recursion

Follow-up: N-Queens I (#51) — same backtracking but must reconstruct and return boards.

Follow-up: count solutions for an n×n board with pre-placed queens — seed initial masks.

IN YOUR APP | Expose | Editorial | Community

HARD | Backtracking | LeetCode #126 | 4 of 6 | 4/6

Word Ladder II

QUESTION

A transformation sequence from word beginWord to word endWord using a dictionary wordList is a sequence of words beginWord -> s_1 -> s_2 -> ... -> s_k such that:

CONSTRAINTS

- Every adjacent pair of words differs by a single letter.
- Every s_i for 1 <= i <= k is in wordList. Note that beginWord does not need to be in wordList.
- s_k == endWord

Given two words, beginWord and endWord, and a dictionary wordList, return all the shortest transformation sequences from beginWord to endWord, or an empty list if no such sequence exists. Each sequence should be returned as a list of the words [beginWord, s_1, s_2, ..., s_k].

Example 1

Input	`beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]`
Output	`[["hit","hot","dot","dog","cog"],["hit","hot","lot","log","cog"]]`
Explanation	*There are 2 shortest transformation sequences: "hit" -> "hot" -> "dot" -> "dog" -> "cog" "hit" -> "hot" -> "lot" -> "log" -> "cog"*

Example 2

Input	`beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log"]`
Output	`[]`
Explanation	*The endWord "cog" is not in wordList, therefore there is no valid transformation sequence.*

- 1 <= beginWord.length <= 5
- endWord.length == beginWord.length
- 1 <= wordList.length <= 500
- wordList[i].length == beginWord.length
- beginWord, endWord, and wordList[i] consist of lowercase English letters.
- beginWord != endWord
- All the words in wordList are unique.
- The sum of all shortest transformation sequences does not exceed 10^5.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find all SHORTEST transformation sequences from beginWord to endWord,

changing one letter at a time, each intermediate word in wordList.

Return all such sequences or [] if none. Right?"

"A few quick questions:

endWord must be in wordList? Yes. beginWord doesn't need to be? Correct.

Words are all lowercase same length? Yes."

"Let me walk: beginWord='hit', endWord='cog', words=['hot','dot','dog','lot','log','cog'].

hit->hot->dot->dog->cog and hit->hot->lot->log->cog -> both length-5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(word, path):
    if word == end:
        record path
    for away in aways:
        dfs(it, path + it)
        best = min(best, recorded_paths)
```

TIME	SPACE
exponential	O(N * L)

WHY IT IS SLOW

Explores every simple path through the word graph. A BFS computes each word's distance from the start, then walking back along edges that go one step closer finds only the shortest paths.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is rebuilding all paths via backtracking.

BFS + parent map + DFS reconstruction is already optimal:

BFS builds the shortest-path DAG (Directed Acyclic Graph) (parents map).

DFS backward from endWord reconstructs all shortest paths.

Removing words from word_set level-by-level prevents revisiting.

Time: O(n*L*26) BFS + O(paths*L) DFS. Space: O(n*L) parents map.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _126_WordLadderII:
    def findLadders(self, beginWord, endWord, wordList):
        from collections import defaultdict
        word_set = set(wordList)
        if endWord not in word_set: return []
        parents = defaultdict(set)
        current_level = {beginWord}
        found = False
        while current_level and not found:
            word_set -= current_level
            next_level = set()
            for word in current_level:
                for i in range(len(word)):
                    for c in 'abcdefghijklmnopqrstuvwxyz':
                        nxt = word[:i] + c + word[i+1:]
                        if nxt in word_set:
                            next_level.add(nxt)
                            parents[nxt].add(word)
                            if nxt == endWord:
                                found = True
            current_level = next_level
        if not found: return []
        result = []
        def dfs(word, path):
            if word == beginWord: result.append(path[::-1]
); return
            for parent in parents[word]: dfs(parent, path
+ [parent])
        dfs(endWord, [endWord])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace findLadders('hit','cog',['hot','dot','dog','lot','log','cog'])."

BFS from 'hit' to find shortest path length AND parents map.

Level 0: {'hit'}. Level 1: change each char: hit->h_t->hot in words -> {'hot'}.

Level 2: hot->h_t->hit(visited),dot,lot -> {'dot','lot'}.

Level 3: dot->dog,lot->log -> {'dog','log'}. dog->cog,log->cog -> {'cog'}.

Path length = 4. parents: cog<-{dog,log}, dog<-{dot}, log<-{lot}, dot<-{hot}, lot<-{hot}, hot<-{hit}.

DFS backwards from 'cog': cog<-dog<-dot<-hot<-hit -> ['hit','hot','dot','dog','cog'] ok.

cog<-log<-lot<-hot<-hit -> ['hit','hot','lot','log','cog'] ok.

Return both paths ok

BFS builds layers; backtrack from end->begin via parent map. queue=[(beginWord,0)] during BFS. visited={'beginWord',...} per layer. push unvisited neighbors.

BFS parent map built; backtrack to find paths. path=[endWord]. call(word): if word==beginWord->save path. For pred in parents[word]: path.append(pred), recurse, backtrack (pop). prune: skip already-visited layers.

PHASE 6 COMPLEXITY & FOLLOW-UP

BFS O(n*L*26). DFS O(paths*L). Space O(n*L) parents.

The level-set removal is critical — without it, parents can include same-level connections.

Follow-up: Word Ladder I (#127) — return just the length; BFS alone suffices.

Follow-up: bidirectional BFS meets in the middle — roughly halves the BFS search space.

IN YOUR APP | Expose | Editorial | Community

HARD | Backtracking | LeetCode #301 | 5 of 6 | 5/6

Remove Invalid Parentheses

QUESTION

Given a string s that contains parentheses and letters, remove the minimum number of invalid parentheses to make the input string valid.

Return a list of unique strings that are valid with the minimum number of removals. You may return the answer in any order.

Example 1	
Input	`s = "()())()"`
Output	`["(())()","()()()"]`

Example 2	
Input	`s = "(a())()"`
Output	`["(a())()","(a)()()"]`

Example 3	
Input	`s = ")("`
Output	`[""]`

CONSTRAINTS

- 1 <= s.length <= 25
- s consists of lowercase English letters and parentheses '(' and ')'.
- There will be at most 20 parentheses in s.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Remove the minimum number of parentheses to make the input string valid. Return all unique results. Right?"

"A few quick questions:

Can have letters too (keep them)? Yes. Return in any order? Yes."

"Let me walk: s='()())()' -> ['()()()', '(())()'] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets:
    # of characters to remove, fewest removals first
    if the rest is valid:
        result.append(it)
    stop at the first size that has answers
```

TIME	SPACE
O(2^n * n)	O(n)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

Tries up to 2^n removals and validates each in O(n). Compute how many '(' and ')' must go, then backtrack removing only that many (skipping duplicates), with pruning.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is BFS visiting exponential states.

Count mismatched '(' and ')' — these are the minimum removals.

Backtracking with exact counts: remove exactly min_open '(' and min_close ')'.

Track last removal position to avoid duplicates.

Time: O(2^(min_open + min_close) * n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _301_RemoveInvalidParentheses:
    def removeInvalidParentheses(self, s):
        min_open = min_close = 0
        for ch in s:
            if ch == '(':
                min_open += 1
            elif ch == ')':
                if min_open > 0: min_open -= 1
                else: min_close += 1
        result = set()
        def backtrack(idx, open_count, close_count, rem_op
en, rem_close, path):
            if idx == len(s):
                if rem_open == 0 and rem_close == 0 and op
en_count == close_count:
                    result.add(path)
                return
            ch = s[idx]
            if ch == '(' and rem_open > 0:
                backtrack(idx+1, open_count, close_count,
rem_open-1, rem_close, path)
            if ch == ')' and rem_close > 0:
                backtrack(idx+1, open_count, close_count,
rem_open, rem_close-1, path)
            if ch == '(':
                backtrack(idx+1, open_count+1, close_count
, rem_open, rem_close, path+ch)
            elif ch == ')':
                if open_count > close_count:
                    backtrack(idx+1, open_count, close_cou
nt+1, rem_open, rem_close, path+ch)
            else:
                backtrack(idx+1, open_count, close_count,
rem_open, rem_close, path+ch)
        backtrack(0, 0, 0, min_open, min_close, '')
        return list(result)
```

PHASE 5 TEST & DEBUG

"Let me trace removeInvalidParentheses('()())()') — find min removals."

BFS: level 0 = {'()())()'}. Check validity: not valid (extra ')').

Level 1: remove each char. Generate all strings with 1 removal. Filter duplicates.

Remove each ')' or '(' at each position. e.g., remove index 4 ')' -> '()()()' -> valid ok.

Remove index 0 '(' -> ')())()' -> not valid. Etc.

First level with valid strings: collect all. Return ['()()()','(())()'] ok (2 valid forms).

WHY BFS not DFS: BFS level = number of removals. First level with valid = minimum removals.

WHY deduplicate: removing the first ')' vs second ')' in '()))' gives same string — skip.

Trace: s="()[]{}" -> push '(', see ')' matches top -> pop; push '[', see ']' -> pop; push '{', see '}' -> pop; stack empty -> True

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(2^(rem_open+rem_close) * n)	**O(n * results)**

BFS is cleaner for small n; backtracking with exact counts avoids set overhead.

Follow-up: Minimum Remove to Make Valid (#1249) — just one valid result needed.

Follow-up: Valid Parentheses (#20) — check only, no removal.

HARD | Backtracking | LeetCode #996 | 6 of 6 | 6/6

Number of Squareful Arrays

QUESTION

An array is squareful if the sum of every pair of adjacent elements is a perfect square.

Given an integer array nums, return the number of permutations of nums that are squareful.

Two permutations perm1 and perm2 are different if there is some index i such that perm1[i] != perm2[i].

Example 1	
Input	`nums = [1,17,8]`
Output	`2`
Explanation	*[1,8,17] and [17,8,1] are the valid permutations.*

Example 2	
Input	`nums = [2,2,2]`
Output	`1`

CONSTRAINTS

- 1 <= nums.length <= 12
- 0 <= nums[i] <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

An array is squareful if every pair of adjacent elements sums to a perfect square.

Count the number of distinct squareful permutations of nums.

Two permutations are different if any index differs. Right?"

"A few quick questions:

Can nums have duplicates? Yes — [2,2,2] -> only one distinct squareful perm.

Permutations where the values are identical count as one? Yes."

"Let me walk: nums=[1,17,8] -> [1,8,17] and [17,8,1] -> 2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for ordering in orderings(numbers):
    if every adjacent sum is a perfect square:
        count the distinct ones
```

TIME	SPACE
O(n! * n)	O(n)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

n! orderings. Backtrack, extending only with a number whose sum with the last one is a perfect square, and skip equal values at the same level: far fewer states.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is generating all permutations then filtering.

Backtracking with pruning:

Build permutation one element at a time using a Counter to track available elements.

At each step, only try elements that form a perfect square with the previous element.

Skip duplicate values at the same recursion level.

Time: O(n! / duplicates) with aggressive pruning. Space: O(n) recursion depth."

Key insight: only pairs whose sum is a perfect square form valid adjacencies — building this graph first reduces the backtracking search space dramatically, and skipping duplicate-value branches eliminates symmetric duplicates.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import math
class _996_NumberOfSquarefulArrays:
    def numSquarefulPerms(self, nums):
        from collections import Counter
        count = Counter(nums)
        result = [0]

        def is_perfect_square(n):
            r = int(math.isqrt(n))
            return r * r == n

        def backtrack(prev, remaining):
            if remaining == 0:
                result[0] += 1; return
            seen = set()
            for val in list(count.keys()):
                if count[val] > 0 and val not in seen:
                    if prev is None or is_perfect_square(p
rev + val):
                        seen.add(val)
                        count[val] -= 1
                        backtrack(val, remaining - 1)
                        count[val] += 1
        backtrack(None, len(nums))
        return result[0]
```

PHASE 5 TEST & DEBUG

"Let me trace numSquarefulPerms([1,17,8])."

Check squareful pairs: 1+8=9=3²ok, 1+17=18 not square. 8+1=9ok, 8+17=25=5²ok, 17+8=25ok.

`backtrack(path=[], used=[F,F,F]):`

PHASE 5 TEST & DEBUG

Try 1(i=0): path=[1]. backtrack([1]):

Try 1 again: dup skip. Try 17: is_square(1+17)=18? No -> skip.

Try 8: is_square(1+8)=9? Yes ok. path=[1,8]. backtrack([1,8]):

Try 17: is_square(8+17)=25 ok. path=[1,8,17]. len=3=full -> append ok.

Try 8(i=1): path=[8]. Try 1: is_square(8+1)=9 ok. path=[8,1]. Try 17: 1+17=18 not sq -> skip. backtrack.

Try 17: path=[8,17]. Try 8: already used. Skip.

Try 17(i=2): path=[17]. Try 8: is_square(17+8)=25 ok. path=[17,8]. Try 1: 8+1=9 ok. path=[17,8,1]. append ok.

Return 2 ok ([1,8,17],[17,8,1]).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n! / duplicates) with square-constraint pruning — often much faster	**O(n)** recursion depth + Counter

Follow-up: squareful paths in a graph — build adjacency list of square-summing pairs,

count Hamiltonian paths in O(n! / symmetry).

Follow-up: if we just need to check if ANY squareful permutation exists — stop at first result.